

浙江树人学院信息科技学院 人工智能 期末大作业

(2025/2026学年第二学期)

项目	内容
大作业题目	Void Jack · 虚空21点
专业	物联网工程
班级	物联网 231
姓名	陈宇杰
学号	202305311115

一、项目概述与工具选型

网页项目主题

Void Jack (虚空21点) — 小丑牌美术风格的霓虹赌台式21点扑克游戏。

项目核心功能

- 完整21点规则** — 发牌、要牌 (Hit)、停牌 (Stand)、双倍下注 (Double)、Blackjack 判定, 庄家自动17点规则
- 筹码系统** — 初始\$3000, 支持\$50/\$100/\$200/\$500四档下注, 赢赔2倍, Blackjack赔2.5倍
- 🎭庄家头像 + 玩家可变表情** — 庄家固定🎭浮动动画, 玩家表情随点数实时变化 (😄→😁→😬→😏→😎→😡→😱→😭)
- Canvas粒子特效系统** — 赢牌500+粒子连环爆 (方块爆发+环形火花+金币雨), 发牌火花, 桌面微光, 输牌红闪
- 像素破碎美学** — CRT扫描线、噪声纹理、色差闪烁、卡片边框辉光、爆牌屏幕震动、文字像素分裂
- 赌台桌面** — 绿色台面+菱格纹背景+霓虹边框, 全屏沉浸式体验

7. 键盘快捷键 — H=Hit, S=Stand, D=Double

选用 AI 工具及选择理由

工具名称	选择原因（优势）
Hermes Agent (DeepSeek v4 Pro)	主力协作工具，支持微信端对话。代码生成质量高，能理解复杂需求并一次性生成完整可用的HTML文件。全程通过自然语言提示词驱动开发。
Chrome DevTools (Playwright MCP)	调试工具。AI可直接在浏览器中截图验证效果、检查控制台错误、模拟用户交互，实现"AI写代码→浏览器自动测试"的闭环。

二、第一轮原始提词

创作思路

在完成渐变实验室后，希望再做一个视觉冲击力更强的作品。提出"小丑牌（Balatro）美术风格的21点扑克游戏"的想法——Balatro以暗底+霓虹酸色+CRT扫描线+像素字体+故障动画著称，21点逻辑简单但交互丰富，非常适合展示Vibe Coding的能力。

第一轮提示词

"做一个小丑牌美术风格的21点扑克游戏怎么样"

第一轮成品效果评价

AI确认可行并分析了优势（视觉辨识度高、交互丰富、演示视频效果好），用户确认选题后一次性生成了约350行的完整HTML文件。首版即包含完整游戏逻辑和基本视觉风格，无JS报错。

三、多轮 AI 交互迭代日志

第 1 轮：VoidJack 初版生成

1. 优化需求 用户选择小丑牌风格21点作为第二个作业选题。

2. 优化提示词

"做一个小丑牌美术风格的21点扑克游戏怎么样"

3. AI 返回代码 AI一次性生成了完整HTML文件（约350行），包含暗色CRT扫描线背景、霓虹酸色主题、发牌动画、翻牌特效、爆牌屏幕震动+故障闪烁效果、完整21点规则、筹码系统、键盘快捷键。

4. 本轮优化后页面评价 首版即无JS报错，所有功能正常。视觉风格有Balatro的味道但略显空，赌台感不够强。

第 2 轮：赌台式重构（视角+布局）

1. 优化需求 视图偏小、不够像霓虹赌台、文字可能重合。

2. 优化提示词

"21点也需要修改优化，首先是视图更大一点，整体看起来更像霓虹风格的台桌。然后各种文字显示不要重合。画面风格也要更加贴合小丑牌，多一些像素破碎效果。"

3. AI 返回 全面重写：绿色赌台+菱格纹背景+霓虹边框，95vw全屏布局，卡片80×112px，多层像素特效（噪声/扫描线/色差闪烁/边框辉光），布局分区明确（score badge独立行），按钮四键合一横排。

4. 本轮优化后页面评价 赌台感大幅提升，绿色台面+霓虹边框的对比很有小丑牌的味道。像素破碎效果层丰富但不喧宾夺主。

第 3 轮：粒子特效 + 翻牌动画 + 庄家头像

1. 优化需求 增加粒子效果和动画，卡片更大，给庄家加头像。

2. 优化提示词

"可以再多一些粒子效果和动画效果吗，牌可以再大一点，可以给对面加一个'😏'的头像"

3. AI 返回 卡片放大至100×140px (+56%)。新增😏庄家头像（浮动+故障闪烁动画）。Canvas粒子系统：赢牌500+粒子爆发（方块+环形+金币雨三阶段）、发牌火花、桌面微光。庄家揭牌翻牌动画。爆牌文字像素分裂特效。

4. 本轮优化后页面评价 粒子系统大幅增强了正反馈体验，赢牌时屏幕中央连环爆炸+金币雨的视觉冲击力很强。庄家头像增加了角色感和压迫感。

第4轮：玩家可变表情

1. 优化需求 玩家侧增加随点数变化的表情，输牌时表情没有变化。

2. 优化提示词

"那再给玩家加一个可变的emoji表情，随着手牌点数变化而变化"

3. AI 返回 新增玩家头像系统：≤7→😌, 8-11→😏, 12-14→🎲, 15-17→😞, 18-20→😭, 21→🎲爆牌→🎲红光快速闪烁), 输牌→😞(粉红跳动2.5秒), 赢牌→绿光放大。

4. 本轮优化后页面评价 表情变化让游戏多了情绪层，爆牌时的🎲红光闪烁和输牌时的😭泪脸让失败也有趣味性。

第5轮：粒子增强 + 初始金额调整

1. 优化需求 粒子效果不够明显，正反馈不足；初始金额太少。

2. 优化提示词

"动画粒子效果可以再明显一点，给足玩家正反馈" → "把初始金额加到3000。另外，玩家点数比庄家小导致输的时候表情没有变化"

3. AI 返回 赢牌粒子从~130粒暴增至500+粒（5波×80方块+20方向环形火花+25波金币雨）。初始金额1000→3000。输牌新增😭泪脸+粉红跳动动画。

4. 本轮优化后页面评价 粒子效果质的飞跃，赢牌时眼花缭乱的正反馈非常有满足感。金额调整让游戏更耐玩。

四、调试与测试

测试案例 1：手机端适配验证

步骤	内容
错误现象	需要验证375px宽度下所有功能正常
定位方式	Playwright MCP resize→截图→交互测试
提示词	浏览器自动化为375×812视口，检查所有元素是否可交互
AI 方案	确认零报错、所有按钮可点击、卡片尺寸自适应
最终修复	通过，无需修改

测试案例 2：游戏逻辑完整性

步骤	内容
错误现象	需验证Hit/Stand/Double/Blackjack/爆牌/庄家17点规则全部正确
定位方式	浏览器evaluate→模拟多轮游戏→检查每个场景的结算结果和筹码变化
提示词	AI自主编写测试脚本，逐场景验证
AI 方案	Hit后爆牌→消息BUST→筹码不变；Stand后庄家自动补牌→正确结算；Blackjack→2.5倍赔率
最终修复	全部通过

测试案例 3：粒子系统性能

步骤	内容
错误现象	需确认500+粒子同时渲染不卡顿
定位方式	浏览器evaluate触发burstWin()→检查Canvas帧率和粒子清理逻辑
提示词	检查requestAnimationFrame循环和particles数组是否及时清理
AI 方案	promiseAnimationFrame+life衰减+filter清理，确认60fps流畅
最终修复	通过，无性能问题

五、成品效果展示

网页完整预览

（截图：虚空21点全屏赌台，展示绿色桌面+菱格纹+霓虹边框+庄家🐼头像+玩家表情+筹码显示）

核心交互功能演示

- 下注发牌 — 点击\$100筹码→DEAL按钮→玩家2张牌+庄家1张明牌1张暗牌→表情自动切换
- Hit要牌 — 点击HIT→新牌滑入动画→点数实时更新→表情变化→爆牌时💥红光+屏幕震动+像素分裂
- Stand停牌 — 庄家暗牌翻牌动画→自动补牌至17+→结算→赢牌500+粒子连环爆
- Double双倍 — 追加等额筹码→仅补1张牌→自动停牌结算
- Blackjack — 开局21点→立即结算2.5倍→霓虹闪烁+粒子爆发

六、Vibe Coding 学习总结与思考

1. 使用 AI 协作开发和自己手写代码的区别

与传统开发最大的不同在于**迭代速度的指数级提升**。手写代码时，一个粒子系统从设计到实现可能需要2-3小时（Canvas API查阅、性能调优、动画参数反复试错）；而Vibe Coding中，我只需要说"可以再多一些粒子效果和动画效果吗，给足玩家正反馈"，AI就自动完成了3波→5波、方块+环形+金币雨的多层次粒子系统设计。这让我可以把精力集中在**体验决策**而非**实现细节**上。

但AI不能替代对"好体验"的判断。比如"赢牌粒子应该多大、多密集"这个问题，AI可以给我500个粒子，但"500个是否合适"需要我来感受和判断。Vibe Coding的本质是**AI放大了我的执行能力，但放大不了我的品鉴能力**。

2. 好的提示词对最终成品的影响

本次开发的提示词演进清晰展示了"精确需求→精准输出"的因果关系：

- **方向性提示词** ("做一个小丑牌美术风格的21点") → AI确定了整体视觉方向
- **多维约束提示词** ("视图更大+像霓虹赌台+文字不重合+像素破碎") → AI理解了空间、视觉、细节三个维度的要求
- **体验级提示词** ("给足玩家正反馈") → AI不只是加粒子，而是重新设计了三级爆发（方块/火花/金币雨）

好的提示词不需要写很多字，但需要**精准传达体验目标**。最高效的提示词往往是一句话——"让赢牌爽一点"比"增加3波每波40个粒子"更有效，因为后者剥夺了AI发挥的空间。

3. 本次作业收获与后续改进思路

收获： - 掌握了"体验驱动"的Vibe Coding方法：先定义用户感受（赌台感、压迫感、正反馈），再让AI去实现，比逐像素描述UI更高效 - 学会了用浏览器自动化形成"AI开发→自动验证→截图审查"的闭环，特别是Playwright MCP可以直接在浏览器中测试游戏逻辑 - 认识到分层迭代的价值：第一版搭骨架（游戏规则），第二版做皮（视觉风格），第三版加魂（粒子+表情），每层独立可验证

后续改进思路： - 尝试"反向Prompt"：先让AI描述它理解的视觉效果，确认方向一致后再写代码 - 建立个人"Prompt→效果"对照库，积累有效提示词模式 - 探索 Vibe Coding在游戏开发中的更多可能：音效、连击系统、成就系统等

4. AI 代码生成存在的局限性，人在开发中不可替代的价值

VoidJack的开发过程也暴露了AI的局限：

- **游戏平衡**：AI不会主动判断"\$3000初始+\$500最大下注"的平衡性是否合理，这是我基于"玩得久一点"的体验目标决定的
- **粒子密度判断**：AI给了500个粒子，但"会不会太密、会不会卡"需要人来测试和反馈。第一次粒子偏少，第二次才调到合适的密度
- **情感设计**：爆牌时用☹️还是😬？赢牌时光效偏绿还是偏金？这些决策没有"正确答案"，只有"更好体验"，需要人的审美直觉

人不可替代的价值： 1. **体验架构师** — 定义"好体验"的标准 2. **审美决策者** — 在无数可能性中选出最合适的 3. **迭代引导者** — 判断每次输出是否"还不够"或"已经够了" 4. **玩家角色** — 真正的测试只能由真人完成，AI无法模拟"玩得爽不爽"

Vibe Coding不是让AI替你设计，而是让你用更少的时间实现更多的设计方案。

以上内容基于与AI (Hermes Agent / DeepSeek v4 Pro) 的真实对话记录撰写。